Sprawozdanie z projektu
Symulator transmisji szeregowej RS232

Tomasz Nazar 197613, Filip Pietrzak 198275

30 marca 2026

1 Sformułowanie zadania i założenia szczegółowe

Celem projektu było napisanie aplikacji symulującej transmisję szeregową zgodną ze standardem RS232. Aplikacja została zrealizowana w języku Python z wykorzystaniem biblioteki PyQt6 i demonstruje proces pakowania danych w ramki oraz ich odzyskiwania przez odbiornik. Dodatkowym wymogiem była implementacja mechanizmu dbającego o czystość języka (filtracja "grubiaństw").

1.1 Zasady przekazywania informacji i interfejs

Interfejs podzielono na dwie główne sekcje: Nadajnik (TX) i Odbiornik (RX), które symulują dwa niezależne urządzenia. Jako medium komunikacyjnego użyto wewnętrznego bufora pamięci (zmiennej łańcuchowej), co skutecznie symuluje przepływ bitów między portami.

1.2 Konwersja znak ASCII <-> strumień bitów (RS232)

Zgodnie z protokołem RS232 transmisja odbywa się w sposób asynchroniczny. Przyjęto następujące parametry ramki dla każdego pojedynczego znaku:

- **Długość słowa:** 8 bitów (dla obsługi polskich znaków wymuszono kodowanie `windows-1250`, unikając błędów wynikających ze zmiennej długości znaków w UTF-8).
- **Kolejność bitów:** Od najmniej znaczącego (LSB) do najbardziej znaczącego (MSB).
- **Bity sterujące:** 1 bit startu (zawsze '0') oraz 2 bity stopu (zawsze '11').

Zatem pojedyncza ramka składa się dokładnie z 11 bitów.

1.3 Mechanizm filtracji słów niecenzuralnych

Wprowadzono zewnętrzny plik tekstowy (`slownik.txt`) pełniący rolę bazy słów zakazanych. Aplikacja wczytuje go przy starcie, a następnie za pomocą wyrażeń regularnych (regex) przeszukuje nadawany lub odbierany tekst. W przypadku znalezienia dopasowania, cenzuruje słowo, zastępując jego znaki ciągiem gwiazdek o odpowiadającej długości, ignorując przy tym wielkość liter.

2 Opis przyjętych rozwiązań programowych

2.1 Logika kodowania ramki w Nadajniku

Przed wysłaniem tekst konwertowany jest do surowych bajtów. Następnie z każdego bajtu generowany jest 8-znakowy łańcuch zer i jedynek. Ciąg ten jest odwracany za pomocą wycinków (ang. *slicing*), aby zasymulować wysyłanie od LSB. Na koniec doklejane są bity startu i stopu.

```
# Konwersja na bajty (8 bitow), zapewnia obsluge polskich znakow
byte_data = filtered_text.encode('windows-1250', errors='replace')

encoded_stream = ""
for byte_val in byte_data:
    # Formatowanie liczby do 8-bitowego ciagu binarnego
    bin_str = format(byte_val, '08b')

    # Odwrocenie kolejnosci: LSB jako pierwsze
    lsb_first = bin_str[::-1]

    # Utworzenie 11-bitowej ramki
    frame = '0' + lsb_first + '11'
    encoded_stream += frame
```

Listing 1: Formatowanie bajtów do ramki RS232

2.2 Filtracja profanacji z użyciem modułu `re`

Do implementacji cenzury użyto biblioteki `re`. Użycie `re.IGNORECASE` pozwala na wyłapywanie słów wpisanych różną wielkością liter, a funkcja `sub` dynamicznie zamienia rdzeń na gwiazdki.

```
def filter_profanity(self, text):
    filtered_text = text
    for word in self.bad_words:
        # Kompilacja wzorca ignorujacego wielkosc liter
        pattern = re.compile(re.escape(word), re.IGNORECASE)
        # Zamiana slowa na gwiazdki o tej samej dlugosci
        filtered_text = pattern.sub('*' * len(word), filtered_text)
    return filtered_text
```

Listing 2: Zastosowanie wyrazen regularnych do cenzury

3 Dyskusja osiągniętych wyników (Wady i Zalety)

3.1 Zalety napisanej aplikacji

- **Wierność logiczna protokołu:** Algorytm w poprawny sposób odwraca kolejność przesyłanych bitów (LSB → MSB) oraz rygorystycznie dokleja asynchroniczne bity sterujące, co stanowi doskonałe odwzorowanie logiki RS232.
- **Odporność na znaki diakrytyczne:** Odkrycie i rozwiązanie problemu wielobajtowego kodowania znaków (UTF-8) poprzez wymuszenie jednobajtowego kodowania `windows-1250`. Pozwoliło to zachować ścisłą długość 11 bitów na ramkę nawet dla polskich znaków.

- **Detekcja błędów ramki:** Odbiornik weryfikuje, czy pierwszy bit to '0', a dwa ostatnie to '11'. W razie desynchronizacji (lub wstrzyknięcia błędu), aplikacja oznacza uszkodzony znak symbolem zapytania.

3.2 Wady i pole do ulepszeń

Mimo zachowania założeń logicznych, symulacja programowa posiada ograniczenia w stosunku do rzeczywistego interfejsu sprzętowego:

- **Brak asynchroniczności czasowej (Baud Rate):** W prawdziwym standardzie RS232 bity przesyłane są w domenie czasu ze ściśle określoną prędkością (np. 9600 bps). W aplikacji cały zdekodowany łańcuch przekazywany jest blokowo, w ułamku sekundy.

- **Brak warstwy elektrycznej:** Program symuluje stany logiczne za pomocą znaków '0' i '1', całkowicie pomijając poziomy napięciowe (standard RS232 to typowo od -15V do +15V) oraz zagadnienia szumów czy tłumienia na przewodach.

- **Medium transmisyjne:** Wykorzystanie zmiennej w pamięci programu jako łącza sprawia, że komunikacja jest w 100% bezstratna. W celu ulepszenia symulatora można zaimplementować celowe, losowe zakłócanie (odwracanie) bitów w locie lub użyć komunikacji sieciowej za pomocą lokalnych gniazd (ang. *sockets*).